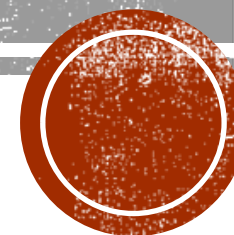


# PILAS

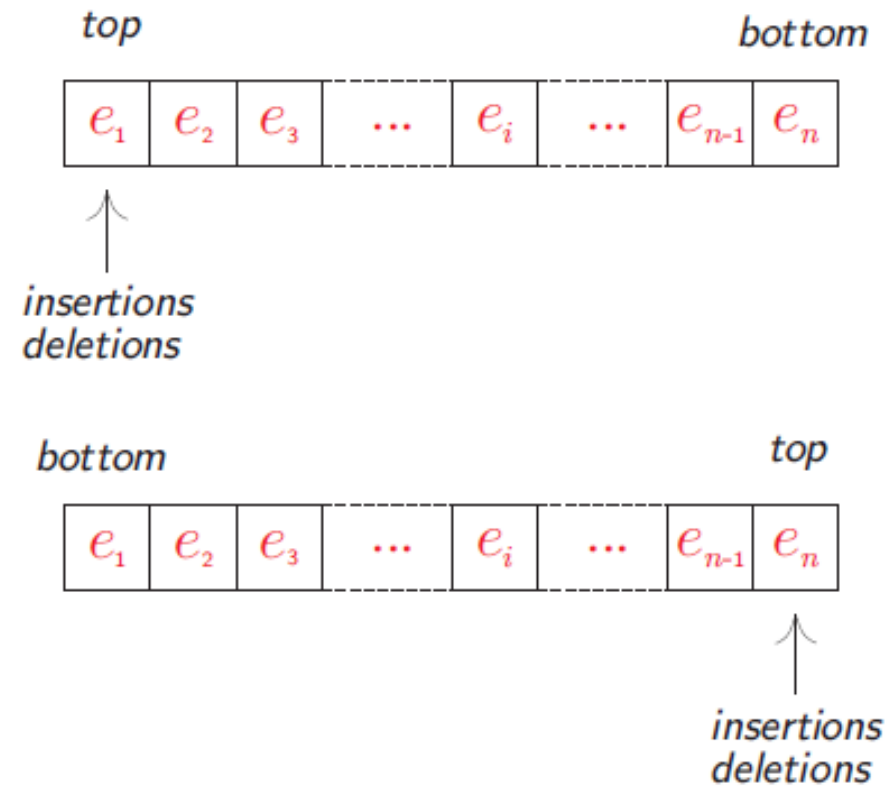
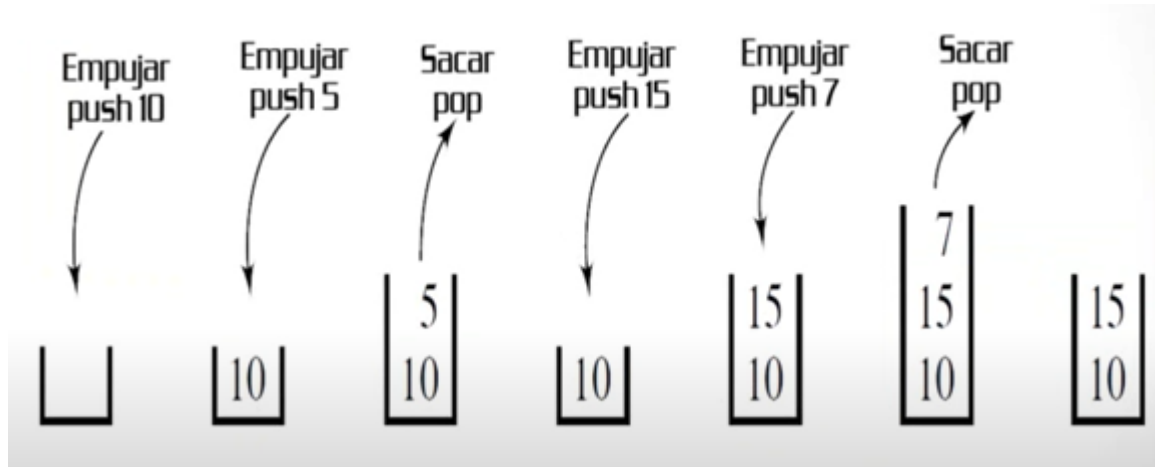


# PILAS

- Una pila (stack) es una colección ordenada de elementos a los cuales solo se puede acceder por un único lugar o extremo de la pila. Los elementos se añaden o se borran de la pila solo por su parte superior (cima). Este es el caso de una pila de platos, una pila de libros, etc.
- En otras palabras, una pila es LIFO (Last In First Out - último en entrar es el primero en salir)

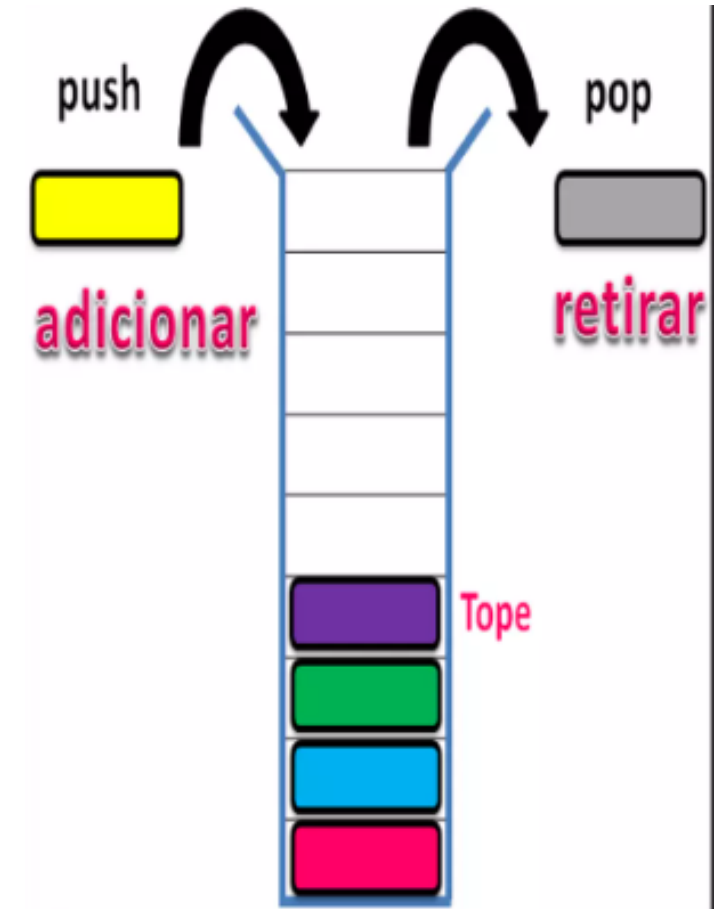


# ASÍ FUNCIONA UNA PILA



# OPERACIONES BÁSICAS CON UNA PILA

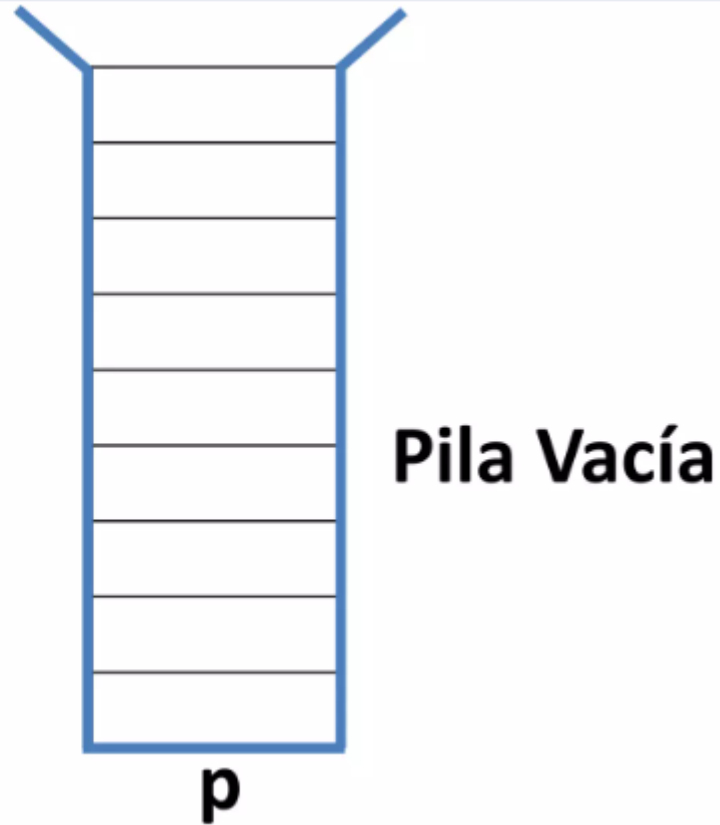
- Crear pila (constructor)
- Insertar dato → push () añade elem. A la pila
- Quitar dato → Pop () Lee y retira elem. superior
- Pila vacía → empty () true si la pila está vacía
- Pila llena
- Limpiar pila
- Cima pila → peek() lee elem. Superior sin retirarlo
- Tamaño de la pila (size) → Regresa # elem. De la pila



# CREAR

**Crear (constructor):**  
crea una pila vacía.

**Stack p = new Stack();**

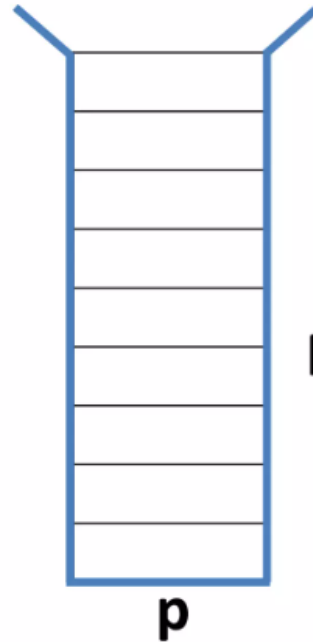


# TAMAÑO

**Tamaño (*size*):** regresa el número de elementos de la pila.

```
int n = p.size();
```

**n = 0**

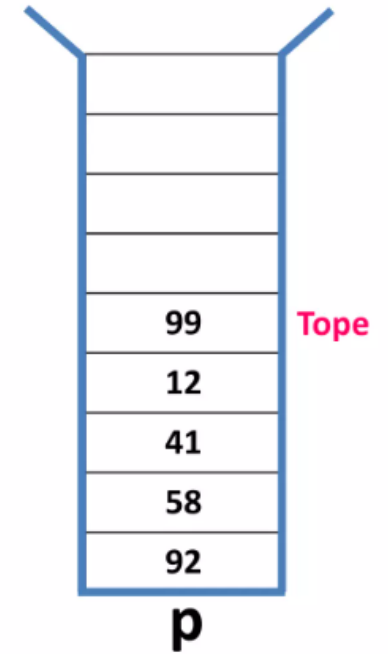


Pila Vacía

**Tamaño (*size*):** regresa el número de elementos de la pila.

```
int n = p.size();
```

**n = 5**



# ADICIONAR

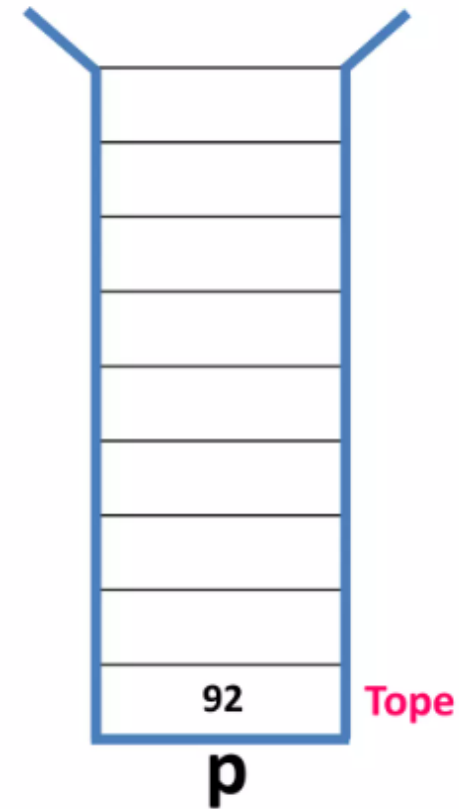
**Apilar (*push*):**

añade un elemento a la pila.

```
int c = 92;
```

```
Integer dato = new Integer(c);
```

```
p.push(dato);
```

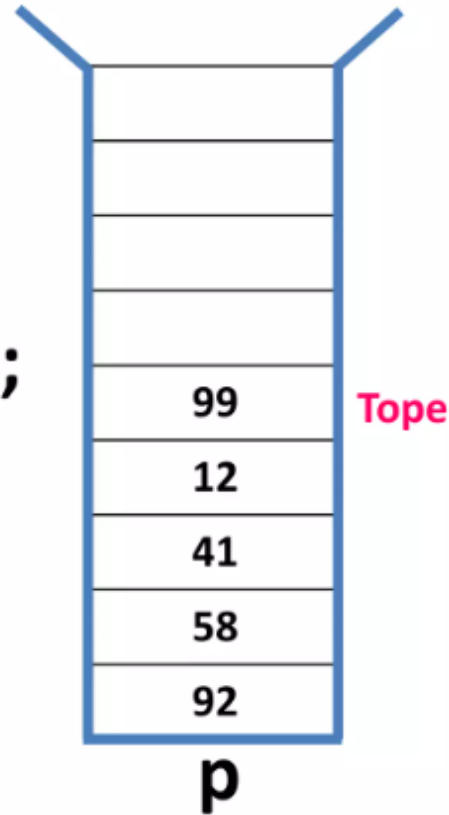


# SACAR

**Desapilar (*pop*):** lee y retira el elemento superior de la pila.

```
int x;  
Integer dato = (Integer )p.pop();  
x = dato.intValue();
```

**x = 99**



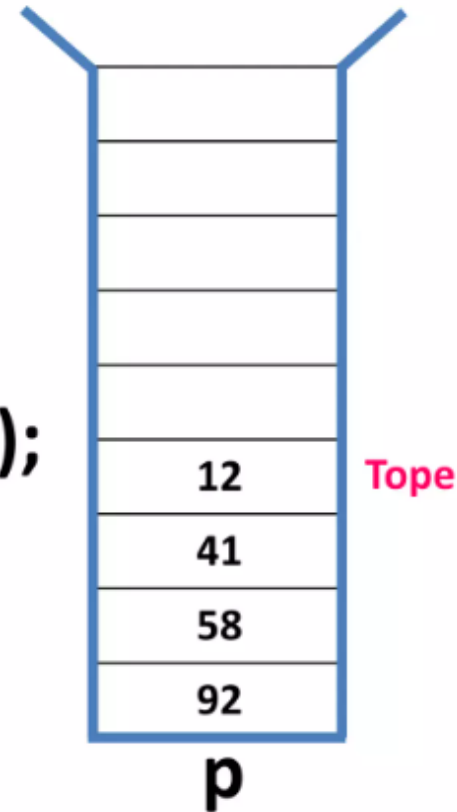


# LEER ÚLTIMO

**Leer último (*top* o *peek*):**  
lee el elemento superior de la pila  
sin retirarlo.

```
int x;  
Integer dato = (Integer)p.peek();  
x = dato.intValue();
```

**x = 12**

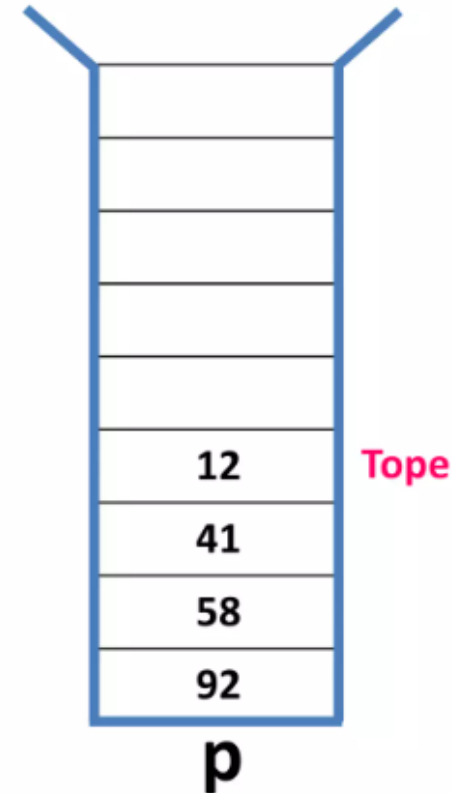


# ¿ESTÁ VACÍA?

**Vacía (*empty*):** devuelve cierto si la pila está sin elementos o falso en caso de que contenga alguno.

```
boolean esta;  
esta = p.empty();
```

**esta = false**



# REPRESENTACIONES DE UNA PILA

Memoria  
Estática

Arreglos  
(Vectores)

Memoria  
Dinámica

- ArrayList
- Nodos



# COINCIDENCIA DE PARÉNTESIS

- Escanear la expresión de izquierda a derecha
- Cuando se encuentre un paréntesis izquierdo, agregue su posición en la pila
- Cuando se encuentre un paréntesis derecho, vuelva a mover la posición coincidente desde la pila.

↓ ↓ ↓

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24  
(((a+b)\*c+d\*e)/(f+g)-h))

2  
1  
0

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓  
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24  
(((a+b)\*c+d\*e)/(f+g)-h))

(2 6) (1 13) (16 20) (15 23) (0 24)

